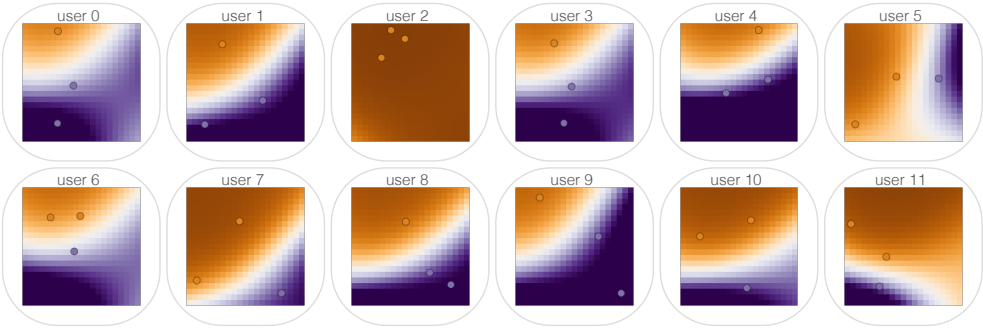
How Federated Learning Protects Privacy



Twelve people periodically merge their models to collaboratively train a shared model — all without sharing their data.

Local Steps: 154
Federated Rounds: 000

Large datasets have made astounding breakthroughs in machine learning possible. But oftentimes data is personal or proprietary, and not meant to be shared, making privacy a critical concern of and barrier to centralized data collection and model training. With **federated learning**, it's possible to collaboratively train a model with data from multiple users without any raw data leaving their devices. If we can learn from data across many sources without needing to own or collect it, imagine what opportunities that opens!

Billions of connected devices — like phones, watches, vehicles, cameras, thermostats, solar panels, telescopes — with sensors to capture data and computational power to participate in training, could collaborate to better understand our environment and ourselves. How do people move? What impacts our health and wellbeing? Together via federated learning these devices could enable new technologies as well. Consider how our cars might contribute to large-scale training of autonomous vehicles without divulging our whereabouts.

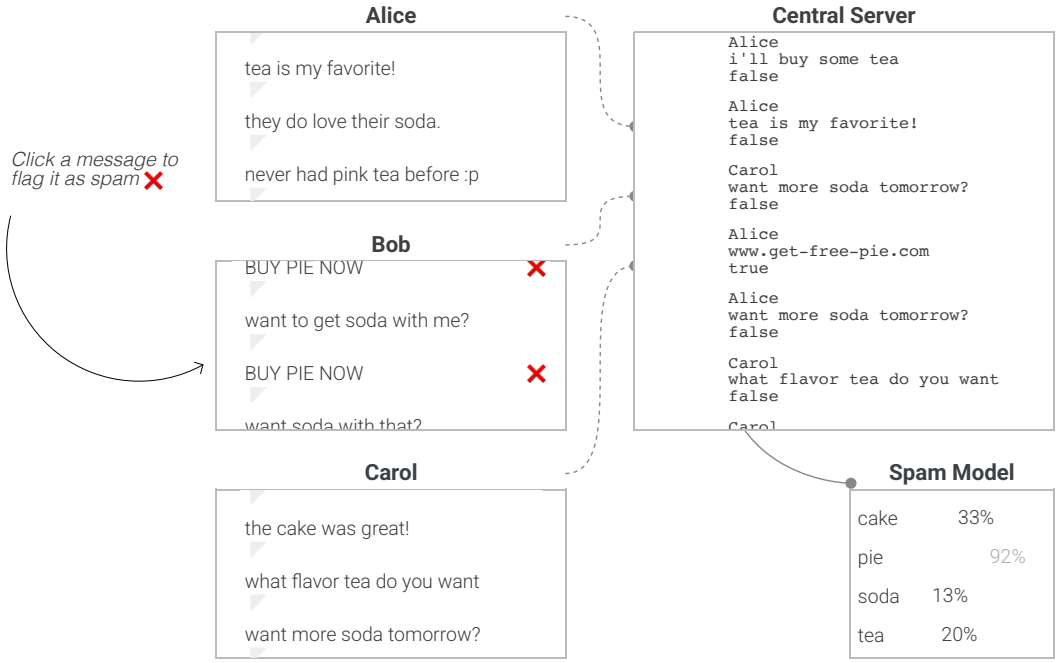
And this machine learning approach can be applied across separate organizations as well. Hospitals could design better treatment plans with knowledge about patient outcomes due to various interventions from care providers worldwide without sharing highly sensitive health data. Pharmaceutical companies with proprietary drug development data could collaborate to build knowledge about how the body is likely to metabolize different compounds.

This framework has the potential to enable large-scale aggregation and modeling of complicated systems and processes like urban mobility, economic markets, energy use and generation patterns, climate change and public health concerns. Ultimately, the hope of federated learning is to allow people, companies, jurisdictions and institutions to collaboratively ask and answer big questions, while maintaining ownership of their personal data.

Designing a Federated Learning System

Let's explore how this technology works with a simple example we can all relate to: blocking spam messages. Spam in chat apps is annoying and pervasive. Machine learning offers a solution – we could develop a model that automatically filters out incoming spam based upon what users previously marked as spam on their devices. This sounds great, but there's a catch: most machine learning models are trained by collecting vast amounts of data on a central server; and user messages can be quite personal. To protect privacy, is it possible to train a spam detection model — or any machine learning model, for that matter — without sharing any potentially sensitive information with a central server?

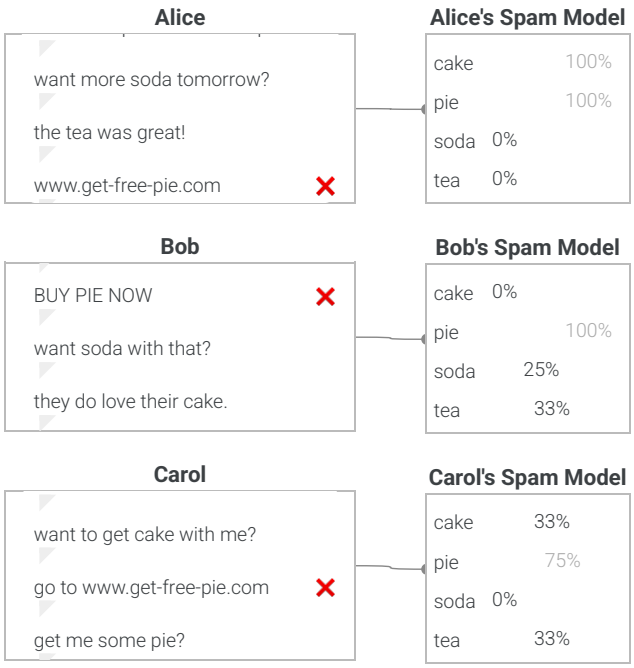
To answer this question, let's first take a closer look at a typical centralized training system, illustrated by the simple spam detection model below. User messages are uploaded to a central server, where they're processed all at once to train a bag-of-words model. Click a message to flag it as spam ✕ or not to change the data uploaded to the server and the trained model.



With centralized training, each user's data is uploaded to a server where a single model is trained.

This model might be pretty good at filtering out spam messages. But centralized training comes with a big downside: all the messages, no matter how sensitive, need to be sent to the server, requiring users to trust the owners of that centralized server to protect their data and not misuse it.[1]

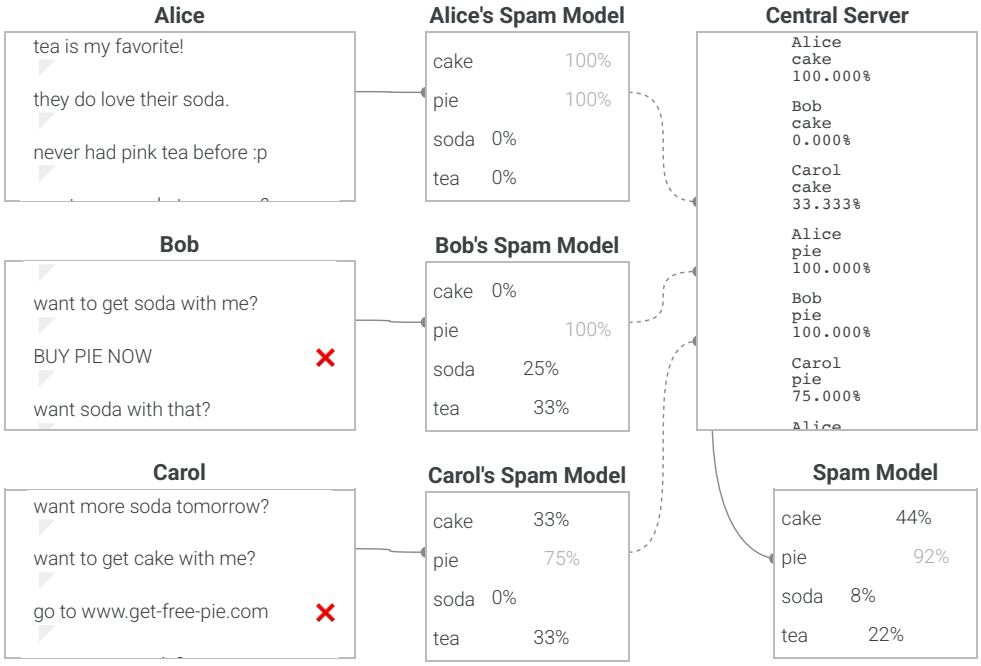
What if training was done locally on each user's device instead and their data wasn't centrally collected? Smartphones are getting increasingly powerful, and they're often idle — for example, while charging overnight — enabling machine learning model training to run without impacting the user experience.



Each user **locally trains** a spam model independently — no information is shared with a server.

Training models locally is great for privacy — no data ever leaves a user's device! — but we can see here how a single device with limited data might not be able to train a high quality model. If a new scam involving, say, car insurance starts spamming messages to everyone, Alice's phone wouldn't be able to filter out messages about "your auto warranty renewal" with a local-only model until she marks several of them as spam — even if Bob has already flagged similar messages.

How can users help each other out and collaboratively train a model without sharing their private data? One idea is for users to share their locally trained spam-detection models instead of their messages. The server can then combine these models, for example by averaging them, to produce a global model that everyone could use for spam filtering.



Naively combining models across users produces a shared model but could compromise user privacy.

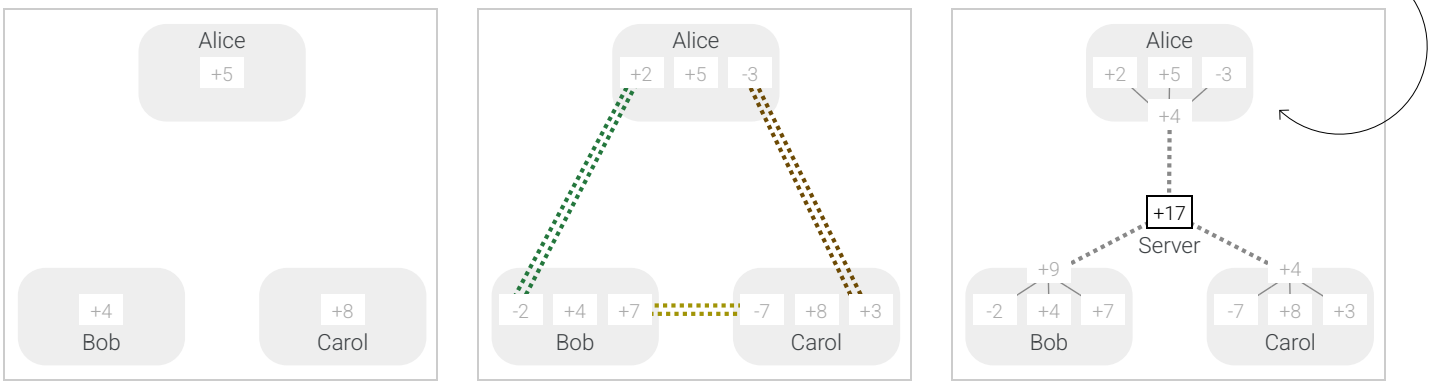
While we've stopped sending every raw message to the server, uploading these local models still leaks some information. Here, the central server has direct access to the rates each user marks different words as spam and can infer what they're talking about. Depending on the level of trust users have in the server, they may be uncomfortable with the server seeing their local models. Ideally the server should only see the aggregated result. We want to develop a system that provides as much data minimization as possible.

Federated learning is a general framework that leverages data minimization tactics to enable multiple entities to collaborate in solving a machine learning problem. Each entity keeps their raw data local, and improves a global model with focused updates intended for immediate aggregation. A good first step towards limiting data exposure when combining user models is to do so without ever storing the individual models — only the aggregate. Secure aggregation and secure enclaves can provide even stronger guarantees, combining many local models into an aggregate without revealing the contribution of any user to the server. This may sound almost magical, so let's take a closer look at how secure aggregation works.

In the secure aggregation protocol, user devices agree on shared random numbers, teaming up to mask their local models in a way that preserves the aggregated result. The server won't know how each user modified their model.

Secure Aggregation: How Does It Work?

Drag numbers to change what's sent to the server



Alice, Bob and Carol want to find the sum of all their hidden numbers

How can they calculate the answer of +17 without anyone revealing their number?

First, Alice & Bob meet in private and pick a shared random number. Alice saves the random number and Bob saves the random number with the sign flipped.

Bob & Carol and Carol & Alice do the same.

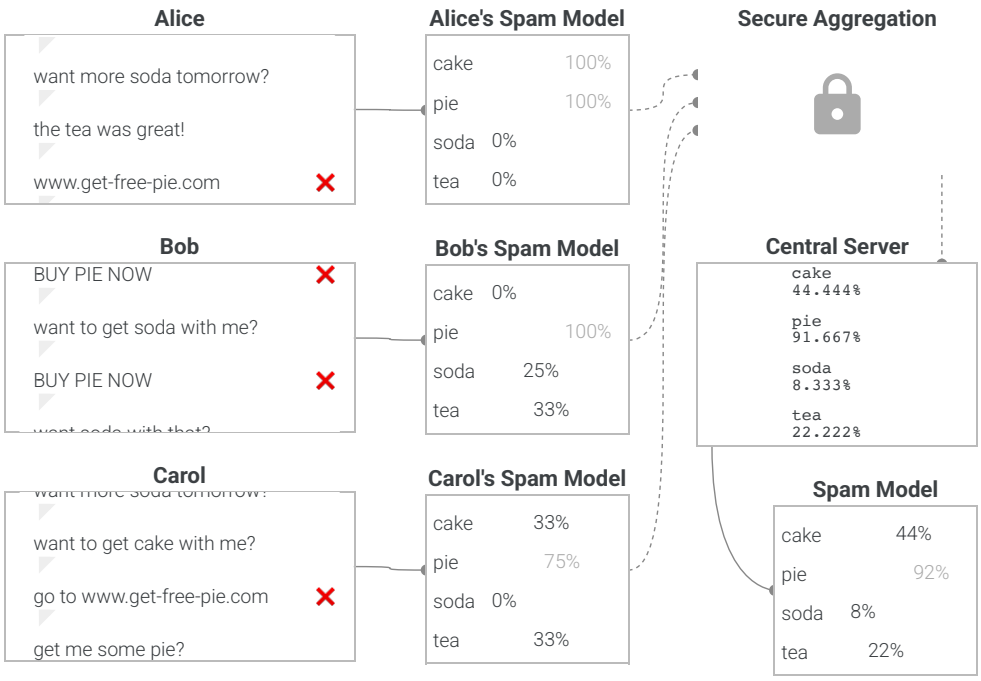
Next, everyone adds up their hidden number and shared random numbers. Then they send the result to a central server.

Finally, the server sums up all results. The pairs of shared random numbers cancel out and the total of the hidden numbers is left over.

Try dragging the shared random numbers — the aggregated sum remains constant even though what

each user sends to the server changes. And importantly, the users' hidden numbers are never shared![2]

Let's put everything together by running all the user-contributed numbers that make up each of their local models through the secure aggregation process. Alice, Bob and Carol's devices use a cryptographic technique to exchange random numbers secretly — our users won't actually meet in person.



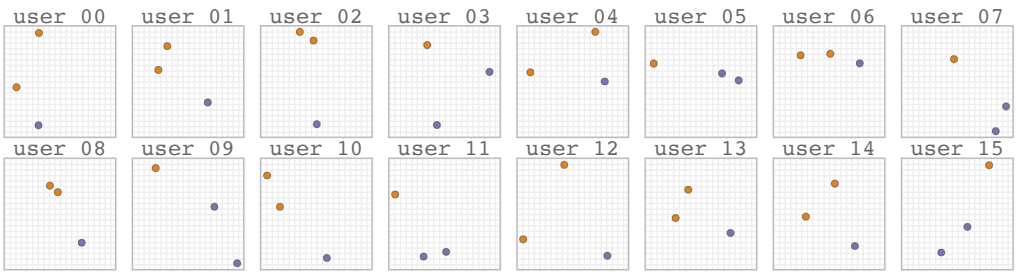
Federated learning — shown here with secure aggegation — produces a shared model while protecting user privacy.

With secure aggregation, users collaboratively merge their models without revealing any individual contribution to the central server.

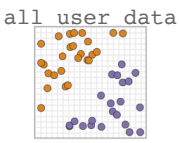
All in all, federated learning enables collaborative model training, while minimizing data exposure. This system design extends to problem settings far beyond the toy spam example we've illustrated above, to larger-scale modeling across all sorts of devices and institutions with privately-held data.

Training a Federated Model

While a very simple model like our toy spam classifier can be learned via a single round of merging local models, more sophisticated models require many iterations of local training and federated averaging. Let's see how that works and examine some challenges that arise in practice. We'll look at a simple "heat-map" binary classification model designed to guess what regions of a grid are likely to be hot or cold. Each of our users has only collected temperature readings from a handful of locations:



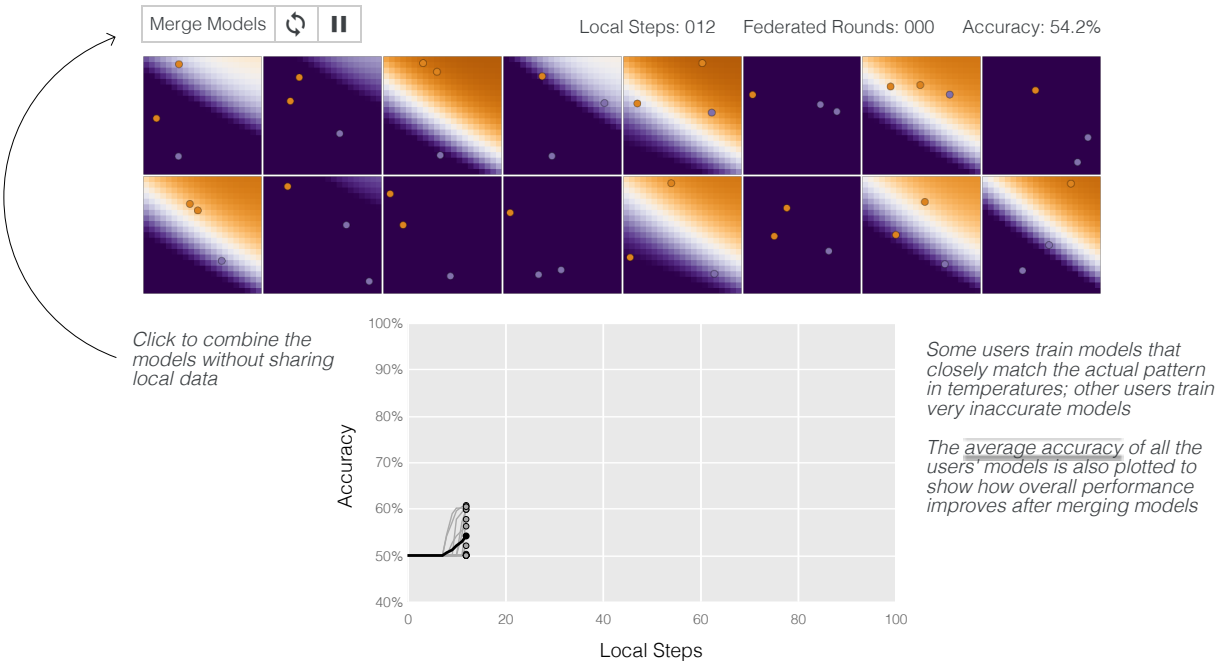
If all the users uploaded their data to a central server, it'd be easy to spot the pattern:



Our goal is to learn this temperature distribution across the grid — so everyone will know where they need

a sweater! — without anyone having to share their location history.

Below, each user is continually training a model with just their local data, predicting the temperature of every location in the grid. You can see how dramatically different models are trained as each user's model overfits to their limited view of the world. The local training curves track the accuracy of each local model on the ground truth data, indicating how well each local model learns the true temperature distribution across the grid.[3]

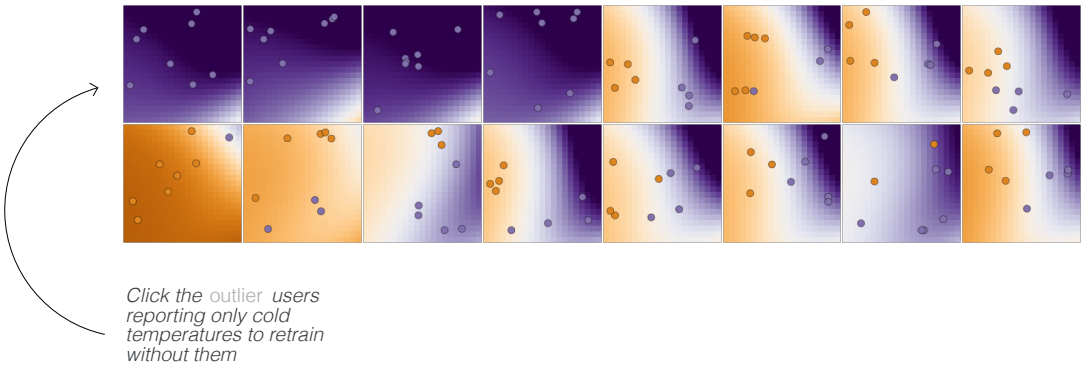


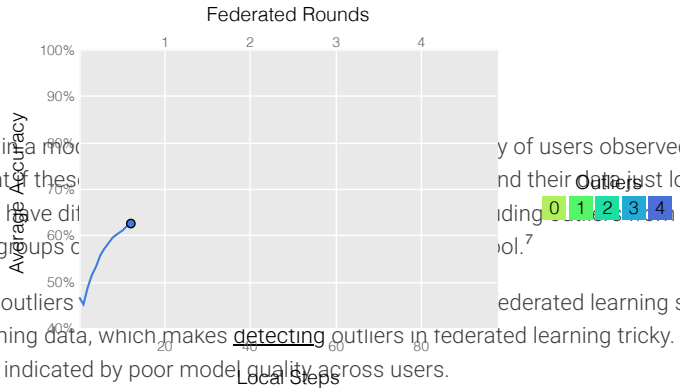
Click Merge Models to run a round of federated training: averaging user models and distributing the updated global model to all users.[4] After training and merging models several times, the resulting global model better resembles the overall temperature distribution across the map than the models trained on just local data. You may notice how local heat map models drift apart after a significant period of local training, and the latest global model's accuracy might degrade upon merging. Relatively frequent periodic averaging is used to avoid this.[5]

While we plot the local model accuracies so it's possible to observe these training dynamics, in practice a server running federated training only has access to the global model. The only metric that can be computed and tracked over the course of training by the server is the global model accuracy.

Outliers

This works pretty well when all users report consistent temperature experiences. What happens if that's not the case? Maybe some of our users have broken thermometers and report cold weather everywhere! Click on each of the four outliers to exclude them from training and notice how the model performs.[6]





We may be able to better train a mo y of users observed without the outliers, but what if thes nd their data just looks different? Some people may have dif ding training risks reducing accuracy for groups o ol.[7]

Though it's easy to spot the outliers ederated learning system cannot directly see user training data, which makes detecting outliers in federated learning tricky. The presence of outliers is often indicated by poor model quality across users.

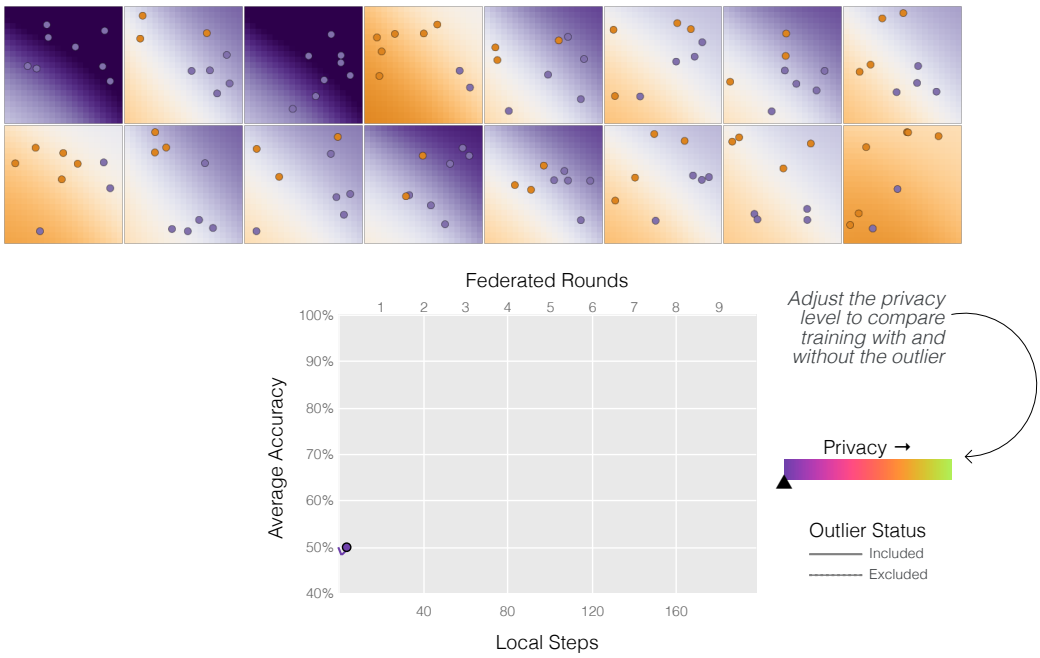
Differential Privacy

Having the global model drastically change based on the presence of a single user also raises privacy concerns. If one user's participation can significantly affect the model, then someone observing the final model might be able to determine who participated in training, or even infer their local data. Outlier data is particularly likely to have a larger impact on model training.

For example, let's say our potential user group includes one person known to always wear a sweater and complain about the cold. If the global model accuracy is lower than expected, we can infer that the notorious sweater-wearing user probably participated in training and reduced accuracy by always reporting cold. This is the case even with secure aggregation — the central server can't directly see which user contributed what, but the resulting global model still gives away that it's likely that a user who believes that it's always sweater weather participated.[8]

Carefully bounding the impact of any possible user contribution and adding random noise to our system can help prevent this, making our training procedure differentially private. When using differential privacy in federated learning, the overall accuracy of the global model may degrade, but the outcome should remain roughly the same when toggling inclusion of the outlier (or any other user) in the training process.

Use the slider to modulate how much the user-reported locations are perturbed. At lower levels of privacy toggling the inclusion of the outlier affects the model more significantly, whereas at higher levels of privacy there is not a discernible difference in model quality when the outlier is included.



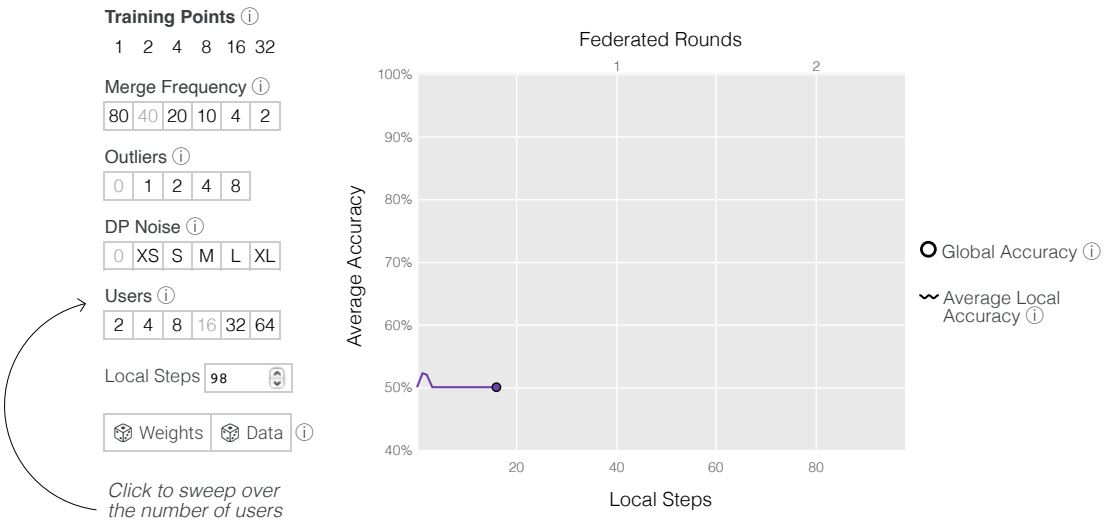
In practice user models are clipped and noised rather than their raw data, or noise is applied to the combination of many clipped models. Applying the noise centrally tends to be better for model accuracy, however the un-noised models may need to be protected by technologies like trusted aggregators.

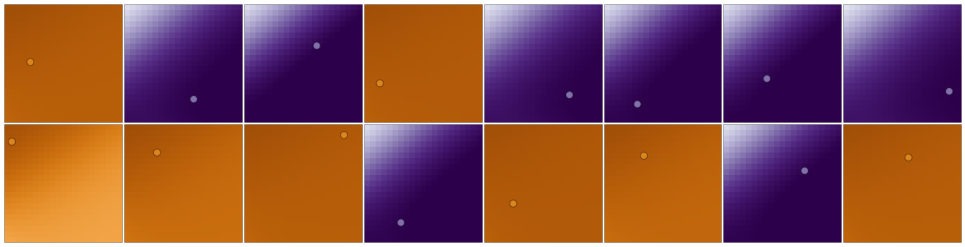
This demonstration illustrates a trade-off between privacy and accuracy, though there's another missing dimension that factors into the equation: the amount of data, both in number of training examples and

number of users. The cost of using more data isn't free — this increases the amount of compute — but it's another knob we can turn to arrive at an acceptable operating point across all of these dimensions.

Playground

There are lots of other knobs to turn in a federated learning setting. All these variables interact in complicated ways. Click on a value for each variable to run a particular configuration, or the variable name to sweep over all of its options. Go ahead and play around — try mixing them together![9]





More Reading

This comic serves as a gentle visual introduction to federated learning. Google AI's blog post introducing federated learning is another great place to start.

Though this post motivates federated learning for reasons of user privacy, an in depth discussion of privacy considerations - namely *data minimization* and *data anonymization* - and the tactics aimed at addressing these concerns is beyond its scope.

Previous explorables have discussed the privacy/accuracy/data trade-off in more detail, with a focus on example-level differential privacy. In many real applications, we care more about user-level differential privacy, which prevents information about any user from being leaked by a published model. Not only is user-level differential privacy stronger than example-level differential privacy, it is quite natural to apply in a federated learning setting since each device has only a single user's data.

There is a wide array of research on Advances and Open Problems in Federated Learning — spanning modeling, system design, network communication, security, privacy, personalization and fairness. Another area of research and development is in federated analytics, which applies the federated framework to answer basic data science questions that do not involve learning without centralized data collection.

If you're interested in trying out federated learning or federated analytics, TensorFlow Federated is an open-source framework you can use. This video series and set of tutorials will help you get started.

Credits

Nicole Mitchell and Adam Pearce // November 2022

Thanks to Nithum Thain, Alex Ingerman, Brendan McMahan, Hugo Song, Daniel Ramage, Peter Kairouz,

Alison Lentz, Kallista Bonawitz, Jakub Konečný, Zachary Charles, Marco Zamarato, Zachary Garrett, Lucas Dixon, James Wexler, Martin Wattenberg, Astrid Bertrand and the Quirk Research team for their help with this piece.

Footnotes

[1] For an end-to-end encrypted messaging application, centralized learning won't work.

[2] More cryptographically sophisticated protocols can be used so a connection between every user isn't required and the sums can still be computed if some users drop out.

[3] Curious how these models are being trained and what's going on in each "local step"? Check out the TensorFlow Playground.

[4] In the cross-device setting (e.g., billions of smartphones), a small fraction of all devices are sampled to participate in each round; typically any one device will contribute to training a handful of times at most. This spreads out the load of training, and ensures the model sees a diversity of different devices. In the cross-silo setting, a small number of larger and more reliable users are assumed (e.g., organizations, datacenters).

[5] In a real cross-device FL training system, each sampled device would generally only compute a fixed relatively small number of local steps before averaging.

[6] The system below merges the local models every 20 local training steps.

[7] Simply including all types of users is often not enough to ensure fairness. Designing strategies for learning a model that performs equally well for everyone is an active area of research. Personalization through local fine-tuning of a final global model is one promising approach.

[8] This might not seem like the biggest deal for this scenario, but participation in medical trials can be highly sensitive and more complex models can leak information.

[9] It's important to remember that this is a simple model and a very small scale federated learning simulation. The phenomena you observe might not be fully representative of what happens in practice.

More Explorables

Collecting Sensitive Information

The availability of giant datasets and faster computers is making it harder to collect and study private information without inadvertently violating people's privacy.

Are Model Predictions Probabilities?

Machine learning models express their uncertainty as model scores, but through calibration we can transform these scores into probabilities for more effective decision making.

Measuring Diversity

Search results that reflect historic inequities can amplify stereotypes and perpetuate under-representation. Carefully measuring diversity in data sets can help.

Can a Model Be Differentially Private and Fair?

Training models with differential privacy stops models from inadvertently leaking sensitive data, but there's an unexpected side-effect: reduced accuracy on underrepresented subgroups.

Hidden Bias

Models trained on real-world data can encode real-world bias. Hiding information about protected classes doesn't always fix things — sometimes it can even hurt.

From Confidently Incorrect Models to Humble Ensembles

ML models sometimes make confidently incorrect predictions when they encounter out of distribution data. Ensembles of models can make better predictions by averaging away mistakes.

What Have Language Models Learned?

By asking language models to fill in the blank, we can probe their understanding of the world.

Measuring Fairness

There are multiple ways to measure accuracy. No matter how we build our model, accuracy across these measures will vary when applied to different groups of people.

Searching for Unintended Biases With Saliency

Machine learning models sometimes learn from spurious correlations in training data. Trying to understand how models make predictions gives us a shot at spotting flawed models.

Can Large Language Models Explain Their Internal Mechanisms?

An interactive introduction to Patchscopes, an inspection framework for explaining the hidden representations of large language models, with large language models.

Datasets Have Worldviews

Every dataset communicates a different perspective. When you shift your perspective, your conclusions can shift, too.

Why Some Models Leak Data

Machine learning models use large amounts of data, some of which can be sensitive. If they're not trained correctly, sometimes that data is inadvertently revealed.

Do Machine Learning Models Memorize or Generalize?

An interactive introduction to grokking and mechanistic interpretability.

AI Explorables Feedback

Đăng nhập vào Google để lưu tiến trình của bạn. Tìm hiểu thêm

What did you like about this explorable? What could be improved? Do you have ideas for future explorables? Get in touch with us!

Câu trả lời của bạn

Your email (optional)

Câu trả lời của bạn

Gửi

Xóa hết câu trả lời

Không bao giờ gửi mật khẩu thông qua Google Biểu mẫu.

Google Biểu mẫu

Biểu mẫu này đã được tạo ra bên trong Google.com.